

Project Report

45 days internship Final Project Report

CodeTrade India Pvt. Ltd.

Submitted by:

Anshula

Project Title: Retrieval-Augmented Generation (RAG) Application using ChromaDB and Google Gemini.

Technology Stack:

- Python
- Sentence Transformers
- ChromaDB
- Google Gemini

Abstract:

Retrieval-Augmented Generation (RAG) is an advanced Natural Language Processing (NLP) architecture that combines semantic document retrieval with Large Language Models (LLMs) to provide context-aware and factually accurate responses. Traditional LLMs rely solely on pre-trained knowledge, making them prone to hallucinations when answering domain-specific or private knowledge questions.

This project implements a complete RAG pipeline for the fictional healthcare organization AuraHealth Nexus. The system loads internal documents, converts them into vector embeddings using Sentence Transformers, stores the embeddings inside ChromaDB, retrieves the most relevant information based on user queries, and finally generates responses using Google's Gemini LLM.

The generated answers are strictly based on the retrieved context, significantly reducing hallucinations while improving answer accuracy.

Table of Content:

1. Introduction
2. Problem Statement
3. Objectives
4. Technologies Used
5. Project Workflow
6. Implementation Details
7. Libraries Used
8. Results
9. Advantages
10. Future Scope
11. Conclusion
12. References

1. Introduction

Large Language Models have transformed artificial intelligence by enabling machines to understand and generate human language. However, they suffer from one major limitation—they cannot access private organizational knowledge unless explicitly provided.

Retrieval-Augmented Generation (RAG) solves this issue by combining vector search with LLMs.

Instead of relying entirely on model memory, the system retrieves relevant information from external documents and provides that information to the language model before generating a response.

This project demonstrates the complete implementation of a RAG pipeline using modern NLP techniques.

2. Problem Statement

The fictional company AuraHealth Nexus possesses highly confidential internal documents containing proprietary medical protocols, research findings, employee policies, and technical specifications.

Since these documents are not part of any public dataset, a language model cannot answer related questions correctly.

The objective is to build a system capable of:

- Reading organizational documents
- Understanding document semantics
- Retrieving the most relevant information
- Answering user queries accurately
- Preventing hallucinated responses

3. Objectives

The primary objectives of this project are:

- Build a Retrieval-Augmented Generation pipeline from scratch.
- Load multiple healthcare documents.
- Split documents into meaningful chunks.
- Generate embeddings for each chunk.
- Store embeddings inside ChromaDB.

- Retrieve relevant chunks using semantic similarity.
- Generate answers using Google Gemini.
- Restrict responses strictly to retrieved context.

4. Technologies Used

- Python
- Pandas
- NumPy
- Sentence Transformers
- ChromaDB
- Google Gemini
- Tqdm
- Jupyter Notebook

5. Project Workflow

Documents



Load Documents



Text Cleaning



Chunking



Sentence Embeddings



ChromaDB Storage



User Query



Embedding



Similarity Search



Top-k Chunks



Prompt



Gemini



Answer

6. Implementation details

Module 1

Loading Documents

- Reads all text files
- Stores document contents

Module 2

Chunk Creation

- Splits text into fixed-size chunks
- Maintains overlap for continuity

Module 3

Embedding Generation

Uses

SentenceTransformer('all-MiniLM-L6-v2')

Each chunk becomes a 384-dimensional vector.

Module 4

Vector Database

Uses ChromaDB collection.

Supports:

- Insert
- Delete
- Similarity Search

Module 5

- Retriever
- Converts user query into embedding.
- Retrieves top matching chunks.

Module 6

Generator

Google Gemini receives:

- User question
- Retrieved chunks

Returns final answer.

7. Libraries Used

- Pandas → Data manipulation.
- NumPy → Numerical Operations.
- Sentence Transformers → Semantic embeddings.
- ChromaDB → Vector similarity search.
- Google Generative AI → LLM inference.
- Tqdm → Displays execution progress.

8. Results

The developed RAG system successfully answers all evaluation questions by retrieving relevant context from the provided documents.

The retrieval mechanism efficiently identifies the most relevant document chunks, while Gemini generates coherent and context-aware responses.

The system demonstrates:

- High retrieval accuracy
- Reduced hallucination
- Context-based reasoning
- Fast semantic search

9. Advantages

- Highly accurate responses
- Supports private documents
- Easy to update knowledge base
- Reduces hallucinations
- Fast semantic retrieval
- Scalable architecture
- Cost-effective compared to model retraining

10.Future Scope

Possible future enhancements include:

- Conversational chat history
- Hybrid Search (BM25 + Vector Search)
- FAISS implementation
- Streamlit Web Application
- PDF support
- OCR support
- Metadata filtering
- Multi-user support
- Incremental indexing
- Real-time document updates

11.Conclusion

This project successfully demonstrates the implementation of a Retrieval-Augmented Generation system using modern NLP technologies.

By integrating Sentence Transformers, ChromaDB, and Google's Gemini model, the application retrieves relevant organizational information and generates accurate, context-aware responses.

Unlike conventional language models, this system minimizes hallucinations by grounding every answer in retrieved document content. The modular design also makes it adaptable to other domains such as healthcare, finance, education, legal services, and enterprise knowledge management.

Overall, the project highlights how RAG can significantly improve the reliability and usefulness of AI-powered question-answering systems.

12. References

- [OpenAI](#)
- [ChromaDB Documentation](#)
- [Google Gemini API Documentation](#)
- [Python documentation](#)